

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

*



BÁO CÁO MÔN HỌC IT3160
WORD-LEVEL HANDWRITTEN TEXT RECOGNITION
NHẬN DẠNG CHỮ VIẾT TAY MỨC ĐỘ TỪ ĐƠN

Giảng viên phụ trách: TS. Ngô Văn Linh

Mã lớp: IT3160

Danh sách sinh viên:

STT	Họ tên	Mã sinh viên	Lớp
1	Nguyễn Trí Hiếu	202416203	CTTN KHMT K69
2	Đỗ Hải Đăng	202400035	CTTN KHMT K69
3	Đinh Thái Sơn	202416746	CTTN KHMT K69

Hà Nội, tháng 6 năm 2026

TÓM TẮT BÁO CÁO

Báo cáo trình bày quá trình xây dựng hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ sử dụng PyTorch. Hệ thống nhận đầu vào là ảnh chứa một từ tiếng Anh viết tay và trả về chuỗi ký tự tương ứng. Dự án được huấn luyện và đánh giá trên bộ dữ liệu IAM Handwriting Dataset ở mức word-level, bao gồm 111,706 mẫu sau khi làm sạch và mở rộng từ vựng hỗ trợ đầy đủ chữ thường, chữ hoa, số cùng các ký tự đặc biệt (Phase 2).

Trong khuôn khổ môn học, nhóm thực hiện xây dựng và đối chiếu hai kiến trúc chính: mô hình đối chứng CTC Baseline (sử dụng ResNet18 + BiLSTM + CTC Loss) và mô hình đề xuất Attention Model (sử dụng ResNet18 + Context BiLSTM + Bahdanau Attention + GRU Decoder). Kết quả thực nghiệm trên tập kiểm thử IAM cho thấy mô hình CTC Baseline đạt 79.95% Word Accuracy và 8.25% Character Error Rate (CER). Trong khi đó, mô hình Attention đạt kết quả tốt hơn với 82.67% Word Accuracy (giải mã Greedy) và tăng lên 82.84% Word Accuracy khi áp dụng giải mã Beam Search. Báo cáo cũng chỉ ra lỗi dừng sớm bằng token <eos> trên tập dữ liệu thực tế và đề xuất hướng khắc phục cụ thể bằng Beam Search.

Mục lục

Tóm tắt báo cáo	i
1 Giới thiệu đề tài	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Mục tiêu và phạm vi thực hiện	1
1.3 Định hướng tiếp cận	2
1.4 Ý nghĩa và cấu trúc báo cáo	2
2 Cơ sở lý thuyết	4
2.1 Bài toán nhận dạng chữ viết tay mức độ từ đơn	4
2.2 Mô hình CTC Baseline	5
2.3 Mô hình Attention nâng cao	6
2.4 Các chỉ số đánh giá hiệu năng	7
3 Dữ liệu và tiền xử lý	9
3.1 Mô tả bộ dữ liệu IAM word-level	9
3.2 Tiền xử lý và tăng cường ảnh	10
3.3 Xử lý nhãn, từ vựng và phân chia dữ liệu	10
4 Kiến trúc mô hình	12
4.1 Trích xuất và biểu diễn đặc trưng	12
4.2 Kiến trúc CTC Baseline	13
4.3 Kiến trúc Attention Model	14
4.4 So sánh và đối chiếu hai kiến trúc	16
5 Cài đặt hệ thống	17
5.1 Môi trường phát triển	17
5.2 Tổ chức mã nguồn	17
5.3 Chuẩn bị dữ liệu và huấn luyện	18
5.4 Đánh giá và suy diễn	19
5.5 Tổng kết chương	20
6 Thực nghiệm và kết quả	21
6.1 Thiết lập thực nghiệm	21
6.2 Kết quả thực nghiệm trên tập kiểm thử IAM	21

6.3	Phân tích kết quả và kiểm thử thực tế	22
7	Kết luận và hướng phát triển	24
7.1	Kết luận và kết quả đạt được	24
7.2	Hạn chế của hệ thống	24
7.3	Hướng phát triển tương lai	25
	Tài liệu tham khảo	26

Danh sách hình vẽ

2.1	Các mức độ khác nhau của bài toán nhận dạng chữ viết tay	4
2.2	Minh họa ý tưởng ánh xạ và căn chỉnh trong mô hình CTC	6
2.3	Cơ chế Attention trong mô hình nhận dạng chuỗi ký tự	7
3.1	Các mẫu ảnh từ viết tay điển hình trong bộ dữ liệu IAM	9
4.1	Sơ đồ luồng xử lý tổng quan của hệ thống HTR	12
4.2	Minh họa quá trình chuyển đổi đặc trưng tích chập 2D thành chuỗi đặc trưng 1D	13
4.3	Kiến trúc chi tiết mô hình CTC Baseline	14
4.4	Kiến trúc chi tiết mô hình Attention	15
5.1	Cấu trúc tổ chức mã nguồn dự án	18
6.1	Ví dụ trực quan hóa attention map của mô hình Attention	23

Danh sách bảng

3.1	Cấu hình kích thước ảnh đầu vào cho hai mô hình	10
3.2	Phân chia bộ dữ liệu IAM sau tiền xử lý (Phase 2)	11
4.1	So sánh kiến trúc CTC Baseline và Attention Model	16
5.1	Cấu hình môi trường thực nghiệm	17
6.1	Kết quả so sánh hiệu năng của hai mô hình trên IAM Test Set (Phase 2) .	21
6.2	Một số ví dụ dự đoán tiêu biểu của hai mô hình	22

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Trong quá trình số hóa tài liệu, nhu cầu chuyển đổi nội dung từ ảnh sang văn bản có thể chỉnh sửa được ngày càng phổ biến. Các hệ thống OCR hiện nay đã xử lý khá tốt nhiều loại văn bản in, biểu mẫu và tài liệu lưu trữ. Tuy nhiên, nhận dạng chữ viết tay vẫn là một thách thức lớn. Nguyên nhân chủ yếu do nét chữ viết tay không có khuôn dạng cố định, thay đổi rất lớn tùy theo thói quen của mỗi người, loại bút sử dụng, tốc độ viết và cả chất lượng của thiết bị chụp ảnh.

Bài toán Word-Level Handwritten Text Recognition (HTR) kết hợp cả xử lý ảnh, trích xuất đặc trưng và mô hình hóa chuỗi ký tự. Khác với các bài toán phân loại ảnh thông thường chỉ chọn một nhãn cố định, hệ thống nhận dạng chữ viết tay phải tự sinh ra một chuỗi ký tự có độ dài biến đổi từ một vùng ảnh đầu vào mà không có thông tin căn chỉnh ký tự cụ thể trước.

Nhóm lựa chọn đề tài này nhằm nghiên cứu và tự xây dựng một pipeline học sâu hoàn chỉnh từ bước xử lý dữ liệu cho đến thiết kế kiến trúc mô hình và giải mã chuỗi. Chúng tôi tập trung so sánh hai hướng tiếp cận kinh điển: mô hình căn chỉnh ngầm sử dụng CTC Loss và mô hình căn chỉnh mềm dựa trên cơ chế Attention. Bộ dữ liệu thử nghiệm chính được chọn là IAM Handwriting Dataset ở mức độ từ đơn.

1.2. Mục tiêu và phạm vi thực hiện

Mục tiêu chính của nhóm là phát triển một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ chạy cục bộ (local) trên PyTorch. Hệ thống nhận đầu vào là một ảnh chứa một từ tiếng Anh viết tay, tiến hành các bước tiền xử lý cần thiết, đưa qua mạng học sâu để trích xuất đặc trưng và dự đoán ra chuỗi ký tự cuối cùng.

Cụ thể, các nhiệm vụ cần thực hiện bao gồm:

- Tìm hiểu và phân tích bài toán nhận dạng chữ viết tay mức từ đơn lẻ.
- Thực hiện tiền xử lý và tăng cường dữ liệu trên tập dữ liệu IAM.
- Triển khai mô hình đối chứng CTC Baseline (ResNet18 + BiLSTM + CTC Loss).

- Triển khai mô hình cải tiến Attention Model (ResNet18 + Context BiLSTM + Bahdanau Attention + GRU Decoder).
- So sánh định lượng hiệu năng của hai mô hình dựa trên các chỉ số chuẩn: Word Accuracy (WA), Character Error Rate (CER), và Normalized Edit Distance (NED).
- Hỗ trợ suy diễn ảnh thực tế, trực quan hóa attention map phục vụ việc giải thích mô hình.

Về phạm vi, dự án chỉ tập trung vào nhận dạng các từ tiếng Anh đơn lẻ viết tay. Ảnh đầu vào được giả định đã được cắt sẵn chứa duy nhất một từ. Chúng tôi không giải quyết các bài toán phức tạp hơn như phát hiện vùng chữ, tách dòng, tách từ hay phân tích bố cục tài liệu hoàn chỉnh. Toàn bộ mã nguồn, quá trình huấn luyện và đánh giá đều được thực hiện local và không gọi bất kỳ API nhận dạng của bên thứ ba nào.

1.3. Định hướng tiếp cận

Quy trình xử lý của hệ thống đi qua các pha: đọc ảnh $\rightarrow$ tiền xử lý ảnh $\rightarrow$ trích xuất đặc trưng không gian bằng mạng CNN (ResNet18) $\rightarrow$ chuyển feature map thành chuỗi đặc trưng $\rightarrow$ học ngữ cảnh bằng BiLSTM và cuối cùng là giải mã chuỗi ký tự. Ở đây, chiều rộng của ảnh sau khi trích xuất được coi như trục thời gian của chuỗi đặc trưng, tương ứng với thứ tự các ký tự viết từ trái qua phải.

Đối với CTC Baseline, mô hình sử dụng hàm mất mát CTC để tự động học sự căn chỉnh giữa ảnh và nhãn mà không cần gán nhãn vị trí từng ký tự. Trong khi đó, với Attention Model, nhóm sử dụng thêm một khối Context BiLSTM để làm giàu thông tin đặc trưng trước khi đưa vào Bahdanau Attention. Cơ chế attention này kết hợp với GRU Decoder để sinh ra từng ký tự một cách tự hồi quy dựa trên các ký tự đã được sinh ở các bước trước đó.

1.4. Ý nghĩa và cấu trúc báo cáo

Dự án giúp nhóm làm quen với thiết kế hệ thống học sâu xử lý chuỗi và hiểu sâu sắc hơn ưu nhược điểm của hai hướng tiếp cận CTC và Attention. Mô hình CTC tuy đơn giản và suy diễn nhanh nhưng thiếu đi khả năng học ngôn ngữ (do giả định các timestep độc lập có điều kiện). Ngược lại, mô hình Attention có khả năng tự hồi quy tốt hơn, biểu diễn trực quan được vùng chú ý qua attention map nhưng đòi hỏi thời gian huấn luyện lâu hơn và dễ bị lỗi dừng sớm hoặc lệch attention trên dữ liệu thực tế ngoài phân phối.

Báo cáo môn học được chia làm 7 chương chính. Chương 1 giới thiệu tổng quan đề tài, mục tiêu và định hướng tiếp cận. Chương 2 trình bày cơ sở lý thuyết về các mô hình mạng và chỉ số đánh giá. Chương 3 mô tả dữ liệu IAM và các bước tiền xử lý ảnh. Chương 4 đi

sâu vào kiến trúc chi tiết của hai mô hình CTC và Attention. Chương 5 mô tả chi tiết việc cài đặt hệ thống local. Chương 6 công bố kết quả thực nghiệm và phân tích lỗi. Chương 7 đưa ra kết luận và vạch ra các hướng phát triển tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Bài toán nhận dạng chữ viết tay mức độ từ đơn

Nhận dạng chữ viết tay (Handwritten Text Recognition - HTR) là quá trình chuyển đổi hình ảnh chứa ký tự viết tay thành văn bản kỹ thuật số tương ứng. Đây là một bài toán trung gian giữa thị giác máy tính và xử lý ngôn ngữ tự nhiên. So với chữ in, chữ viết tay phức tạp hơn nhiều do tính đa dạng trong phong cách viết, nét chữ có thể bị dính nhau, bị nghiêng, mờ hoặc bị đứt gãy tùy thuộc vào thói quen của người viết và loại bút sử dụng.

HTR Task Levels (Mức độ bài toán nhận dạng chữ viết tay)



Hình 2.1: Các mức độ khác nhau của bài toán nhận dạng chữ viết tay

Về mặt toán học, cho ảnh đầu vào X chứa một từ viết tay, mô hình cần tìm chuỗi ký tự dự đoán $Y = (y_1, y_2, \dots, y_L)$ có xác suất điều kiện lớn nhất:

$$\hat{Y} = \arg \max_Y P(Y|X)$$

Trong đó L là độ dài chuỗi ký tự (độ dài này biến thiên tùy theo từ đầu vào). Khó khăn cốt lõi là dữ liệu huấn luyện thường chỉ cung cấp nhãn văn bản của từ chứ không đi kèm vị trí chính xác (căn chỉnh) của từng ký tự trên ảnh. Do đó, mô hình phải tự học cách căn

chỉnh ngầm (qua CTC Loss) hoặc căn chỉnh động (qua cơ chế Attention) trong quá trình tối ưu.

2.2. Mô hình CTC Baseline

Kiến trúc đối chứng CTC Baseline sử dụng một mạng tích chập (CNN) kết hợp với mạng học tuần tự hai chiều (BiLSTM) và hàm mất mát Connectionist Temporal Classification (CTC).

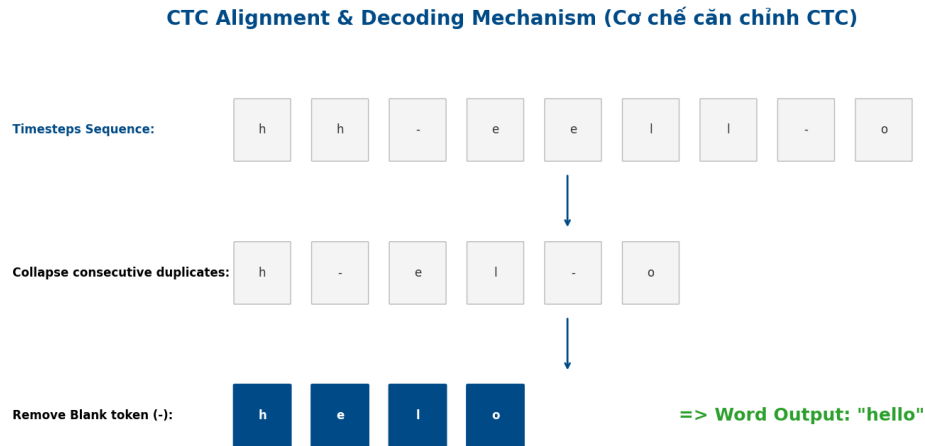
Mô hình sử dụng mạng ResNet18 làm bộ trích xuất đặc trưng (CNN Encoder). Phần đầu vào của ResNet18 nhận ảnh và trả về một feature map hai chiều. Để chuyển đổi đặc trưng không gian thành chuỗi thời gian, feature map này được reshape dọc theo chiều rộng của ảnh (trục thời gian). Với mỗi vị trí theo chiều ngang, vector đặc trưng mô tả một vùng ảnh tương ứng. Sau đó, chuỗi đặc trưng được đưa vào mạng BiLSTM (2 lớp) nhằm khai thác ngữ cảnh hai chiều (trái sang phải và phải sang trái), giúp nhận dạng ký tự chính xác hơn dựa trên các ký tự lân cận.

Hàm mất mát CTC Loss được sử dụng để giải quyết bài toán căn chỉnh dữ liệu. CTC giải quyết vấn đề bằng cách cho phép mô hình sinh ra một chuỗi nhãn trung gian dài hơn nhãn thực, chứa các ký tự và một ký hiệu đặc biệt gọi là blank token (ký hiệu là $-$). Chuỗi trung gian này sau đó được thu gọn bằng cách gộp các ký tự giống nhau liên tiếp và loại bỏ blank token. Ví dụ, chuỗi dự đoán thô $a-a-b-b$ sẽ được rút gọn thành ab .

Hàm loss sẽ tính tổng xác suất của tất cả các đường dẫn trung gian hợp lệ dẫn đến nhãn đúng:

$$P(Y|X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X)$$

Trong đó $\mathcal{B}$ là toán tử thu gọn chuỗi, π là một đường dẫn trung gian. Khi suy diễn, phương pháp giải mã mặc định là CTC Greedy Decoding, chọn ký tự có xác suất cao nhất tại mỗi timestep rồi thực hiện rút gọn. Phương pháp này có ưu điểm là tính toán rất nhanh nhưng không tận dụng được mối tương quan ngôn ngữ giữa các ký tự đã sinh.



Hình 2.2: Minh họa ý tưởng ánh xạ và căn chỉnh trong mô hình CTC

2.3. Mô hình Attention nâng cao

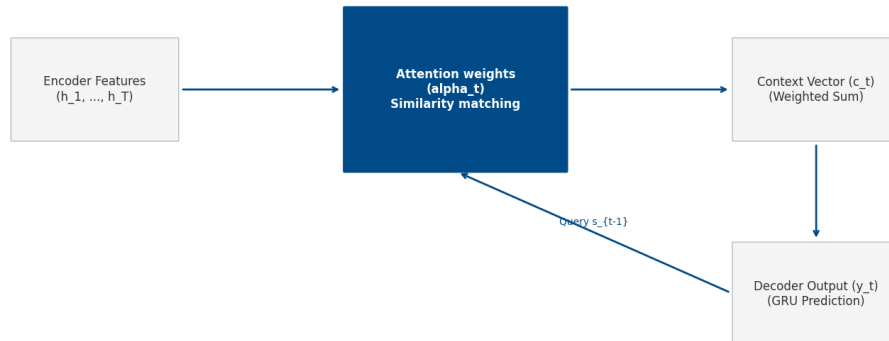
Mô hình đề xuất dựa trên kiến trúc Encoder–Decoder kết hợp với cơ chế Bahdanau Attention. Encoder của mô hình bao gồm một mạng CNN ResNet18 để trích xuất đặc trưng không gian và một Context BiLSTM để làm giàu thông tin ngữ cảnh cho từng vector đặc trưng dọc theo chiều ngang ảnh.

Decoder sử dụng mạng GRUCell để sinh ra chuỗi ký tự từng bước một cách tự hồi quy. Tại mỗi bước giải mã t , thay vì sử dụng một vector đặc trưng cố định, mô hình sử dụng cơ chế Bahdanau Attention để tính toán trọng số chú ý $\alpha_{t,s}$ đối với từng vector đặc trưng h_s từ phía encoder. Điểm tương quan $e_{t,s}$ giữa trạng thái ẩn của decoder s_{t-1} và đặc trưng encoder h_s được định nghĩa:

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Các điểm này được chuẩn hóa qua hàm softmax để tạo thành phân phối trọng số attention $\alpha_{t,s}$, từ đó tính ra vector ngữ cảnh $c_t = \sum_s \alpha_{t,s} h_s$. Trạng thái ẩn của decoder và ký tự tiếp theo được dự đoán dựa trên sự kết hợp của c_t và embedding của ký tự đã sinh ở bước trước.

Bahdanau Attention Mechanism (Cơ chế chú ý Bahdanau)



Hình 2.3: Cơ chế Attention trong mô hình nhận dạng chuỗi ký tự

Trong giai đoạn huấn luyện, kỹ thuật Teacher Forcing được áp dụng nhằm tăng tốc độ hội tụ bằng cách đưa trực tiếp ký tự nhãn đúng (ground-truth) làm đầu vào cho bước tiếp theo thay vì sử dụng dự đoán của bước trước. Tỷ lệ Teacher Forcing được giảm dần (decay) theo từng epoch để giảm bớt sự sai lệch phân phối giữa lúc train và lúc test (Exposure Bias).

Khi suy diễn, mô hình có thể giải mã theo hai cách:

- **Greedy Decoding:** Lấy ký tự có xác suất cao nhất tại mỗi bước thời gian của decoder và lặp lại cho đến khi gặp token kết thúc $\langle \text{eos} \rangle$.
- **Beam Search:** Duy trì đồng thời K chuỗi ứng viên có xác suất tích lũy cao nhất tại mỗi bước thời gian. Kỹ thuật này giúp giảm bớt hiện tượng mô hình bị dừng sớm (do sinh nhầm token $\langle \text{eos} \rangle$ quá sớm) bằng cách giữ lại các nhánh dự đoán dài hơn có điểm số tích lũy tốt.

2.4. Các chỉ số đánh giá hiệu năng

Để đánh giá định lượng hiệu quả nhận dạng của hệ thống, dự án sử dụng ba chỉ số chuẩn trong bài toán OCR và HTR:

1. Word Accuracy (WA): Tỷ lệ phần trăm số từ được mô hình nhận dạng chính xác hoàn toàn 100% trên toàn bộ tập dữ liệu kiểm thử. Chỉ số này rất khắt khe vì chỉ cần sai lệch một ký tự (chữ hoa/chữ thường, thiếu dấu câu...) thì cả từ đó đều bị tính là sai.

$$WA = \frac{\text{Số từ khớp hoàn toàn}}{\text{Tổng số mẫu kiểm thử}} \times 100\%$$

2. Character Error Rate (CER): Tỷ lệ lỗi ở cấp độ ký tự dựa trên khoảng cách chỉnh sửa Levenshtein. Khoảng cách này tính số phép biến đổi tối thiểu (thay thế S , xóa D , chèn I) để chuyển chuỗi dự đoán thành chuỗi đích.

$$CER = \frac{S + D + I}{N} \times 100\%$$

Trong đó N là tổng số ký tự của chuỗi đích. CER cho cái nhìn trực quan và chi tiết hơn WA về khả năng nhận diện nét chữ của mô hình.

3. Normalized Edit Distance (NED): Khoảng cách Levenshtein được chuẩn hóa theo độ dài lớn nhất của chuỗi dự đoán và chuỗi đích nhằm đo đặc độ tương đồng ngữ nghĩa tổng quát giữa hai chuỗi:

$$NED = \left(1 - \frac{1}{M} \sum_{i=1}^M \frac{\text{Levenshtein}(\text{pred}_i, \text{target}_i)}{\max(\text{len}(\text{pred}_i), \text{len}(\text{target}_i))} \right) \times 100\%$$

Chỉ số NED càng tiệm cận 100% thể hiện chuỗi ký tự dự đoán càng gần với nhãn đúng, phản ánh sát sao nỗ lực sửa đổi của mô hình trên từng từ đơn.

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1. Mô tả bộ dữ liệu IAM word-level

Dự án sử dụng bộ dữ liệu IAM Handwriting Dataset ở mức độ từ đơn lẻ (word-level). Đây là bộ dữ liệu chữ viết tay tiếng Anh kinh điển được cung cấp bởi Đại học Bern, chứa các mẫu ảnh cắt ra từ các bài viết tay của nhiều người viết khác nhau. Ở mức độ từ đơn lẻ, mỗi mẫu dữ liệu bao gồm một ảnh chứa duy nhất một từ viết tay và nhãn văn bản đi kèm. Việc sử dụng dữ liệu ở mức độ này giúp hệ thống tập trung hoàn toàn vào mô hình hóa chuỗi ký tự từ ảnh, bỏ qua các bước phức tạp như phát hiện dòng hay tách từ trong câu.



Hình 3.1: Các mẫu ảnh từ viết tay điển hình trong bộ dữ liệu IAM

Dữ liệu thô của dự án được tổ chức thành thư mục chứa tập ảnh và một tập văn bản lưu trữ nhãn:

```
dataset /  
+-- words /  
|   +-- a01-000u-00-00.png  
|   +-- a01-000u-00-01.png  
|   ...  
+-- label.txt
```

Mỗi dòng trong tập `label.txt` ánh xạ tên ảnh với chuỗi văn bản tương ứng.

Tuy nhiên, IAM là một bộ dữ liệu được thu thập trong điều kiện phòng thí nghiệm tiêu chuẩn: ảnh được quét sắc nét, độ tương phản cao, và chữ được cắt tương đối chuẩn xác. Khi thử nghiệm trên ảnh chụp thực tế ngoài đời thực (với nền ảnh tối, đổ bóng, góc chụp nghiêng và nét viết tự do), hiệu năng của mô hình sẽ suy giảm rõ rệt do sự khác biệt lớn về phân phối dữ liệu (domain shift). Đây là một hạn chế quan trọng mà nhóm nhận thấy trong quá trình thực hiện dự án.

3.2. Tiền xử lý và tăng cường ảnh

Ảnh chữ viết tay có độ dài và tỉ lệ khung hình (aspect ratio) biến thiên rất lớn. Do các mô hình học sâu yêu cầu đầu vào dạng batch có kích thước cố định để tối ưu hóa tính toán trên GPU, nhóm đã thiết kế một pipeline tiền xử lý ảnh đồng nhất.

Đầu tiên, ảnh được đọc ở dạng grayscale. Tiếp theo, ảnh được resize về một chiều cao cố định (64 pixel), đồng thời giữ nguyên tỉ lệ khung hình để tránh làm biến dạng chữ viết. Nếu chiều rộng của ảnh nhỏ hơn kích thước đích cấu hình, hệ thống sẽ chèn thêm khoảng trắng (pad giá trị 255) về phía bên phải; ngược lại nếu ảnh quá dài, hệ thống sẽ cắt bớt theo chiều rộng tối đa. Cuối cùng, ảnh grayscale được nhân bản thành 3 kênh để tận dụng trọng số pre-trained của mạng ResNet18 trên ImageNet, rồi được chuẩn hóa theo phân phối chuẩn của ImageNet ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

Bảng 3.1: Cấu hình kích thước ảnh đầu vào cho hai mô hình

Mô hình	Kích thước ảnh	Feature map sau CNN	Độ dài chuỗi thời gian
CTC Baseline	64×512	$(B, 256, 4, 32)$	$T = 32$
Attention Model	64×512	$(B, 256, 4, 32)$	$T = 32$

Trong mã nguồn thực tế, để thuận tiện và đồng bộ cho cả hai pipeline, nhóm thiết lập mặc định chiều rộng ảnh đầu vào là 512 pixel thông qua tệp `config.py`.

Để hạn chế tình trạng quá khớp (overfitting) và giúp mô hình học được các biến dạng chữ viết tay thực tế, các phép tăng cường dữ liệu (data augmentation) được áp dụng cho tập huấn luyện bao gồm: xoay ngẫu nhiên ($\pm 15^\circ$), dịch chuyển, biến đổi affine (shear), biến đổi đàn hồi (elastic transform) để mô phỏng nét viết co giãn, và làm mờ Gaussian ngẫu nhiên. Tăng cường dữ liệu chỉ được áp dụng trên tập huấn luyện, giữ nguyên tập validation và test để đánh giá khách quan.

3.3. Xử lý nhãn, từ vựng và phân chia dữ liệu

Nhãn văn bản thô từ dataset cần được làm sạch và mã hóa thành chuỗi số nguyên để mô hình tính toán loss. Ở phiên bản chính thức (Phase 2), nhóm mở rộng không gian từ vựng để hỗ trợ đầy đủ chữ thường, chữ hoa, số và các ký tự đặc biệt xuất hiện trong IAM.

Bộ từ vựng của Phase 2 bao gồm 76 ký tự: `abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ`
`. / : ; ?`

Mỗi ký tự trong bộ từ vựng được ánh xạ sang một chỉ số nguyên độc nhất. Đối với mô hình CTC Baseline, chỉ số 0 được dành riêng cho blank token (đồng thời là pad token). Với mô hình Attention, hệ thống bổ sung thêm ba token đặc biệt: `<pad>` (chỉ số 0), `<sos>` (chỉ số 1 - ký hiệu bắt đầu chuỗi) và `<eos>` (chỉ số 2 - ký hiệu kết thúc chuỗi). Ký tự đích của Attention Model được thêm `<sos>` ở đầu và `<eos>` ở cuối trước khi đưa qua lớp GRU Decoder.

Sau khi làm sạch nhãn và lọc bỏ các mẫu ảnh lỗi, tổng số mẫu sử dụng cho thực nghiệm Phase 2 là ****111,706 mẫu**** (nhiều hơn con số 96,835 mẫu của Phase 1 do không lọc bỏ các từ chứa chữ số và ký tự đặc biệt). Tập dữ liệu được phân chia ngẫu nhiên theo tỷ lệ 80/10/10 như sau:

Bảng 3.2: Phân chia bộ dữ liệu IAM sau tiền xử lý (Phase 2)

Tập dữ liệu	Số mẫu	Tỷ lệ
Train Set	89,364	80%
Validation Set	11,170	10%
Test Set	11,172	10%
Tổng cộng	111,706	100%

Để nạp dữ liệu hiệu quả dưới dạng mini-batch, nhóm cài đặt lớp `Dataset` và `DataLoader` tùy biến bằng PyTorch. Lớp `Dataset` chịu trách nhiệm đọc ảnh, thực hiện các phép transform và mã hóa nhãn tương ứng. Khi ghép thành batch, do các từ có độ dài ký tự khác nhau, `DataLoader` thực hiện đệm (padding) các nhãn bằng token `<pad>` về cùng độ dài lớn nhất trong batch. Quá trình tiền xử lý, phân chia tập dữ liệu và lưu trữ các tệp CSV chỉ mục được thực hiện một lần duy nhất bằng câu lệnh:

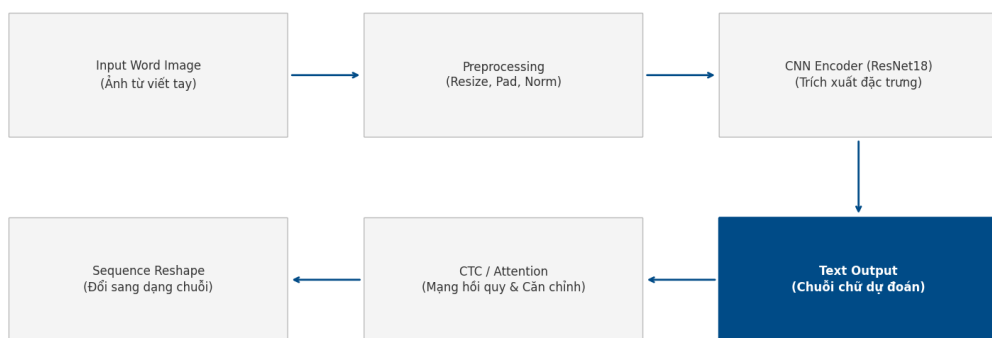
```
python -m src.prepare_data
```

CHƯƠNG 4. KIẾN TRÚC MÔ HÌNH

4.1. Trích xuất và biểu diễn đặc trưng

Hệ thống nhận dạng chữ viết tay mức từ đơn lẻ được xây dựng theo kiến trúc học sâu đầu-cuối (end-to-end). Đầu vào là một ảnh chứa duy nhất một từ viết tay, còn đầu ra là chuỗi ký tự dự đoán tương ứng với nội dung từ trong ảnh.

Overall HTR System Pipeline (Sơ đồ luồng xử lý hệ thống)



Hình 4.1: Sơ đồ luồng xử lý tổng quan của hệ thống HTR

Pipeline tổng quát đi qua các pha chính: CNN Encoder → Sequence Reshape → Sequence Modeling → Decoding.

Ở pha đầu tiên, cả hai mô hình (CTC Baseline và Attention Model) đều sử dụng chung một mạng tích chập ResNet18 pre-trained để trích xuất đặc trưng hình ảnh. Tuy nhiên, thay vì sử dụng toàn bộ mạng phân loại, nhóm chỉ giữ lại phần trích xuất đặc trưng cho đến khối `layer3`. Quyết định dừng lại ở `layer3` thay vì `layer4` (khối cuối cùng của ResNet18) giúp bảo toàn độ phân giải không gian của ảnh theo chiều ngang (trục thời gian W').

Với ảnh đầu vào có kích thước $64 \times 512 \times 3$, feature map đầu ra từ CNN Encoder có dạng

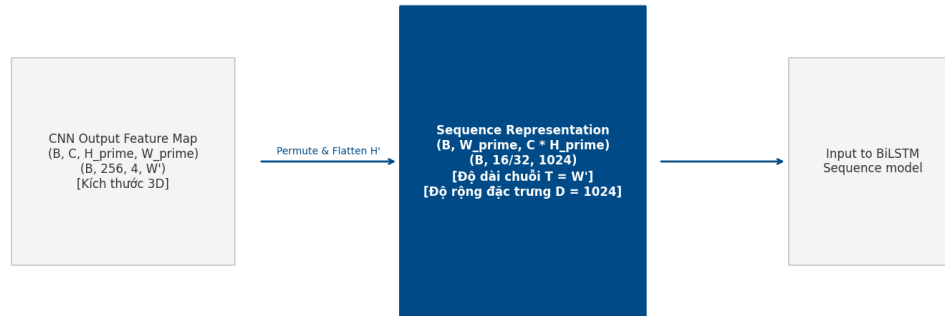
$(B, 256, 4, 32)$, trong đó 256 là số kênh đặc trưng, 4 là chiều cao và 32 là chiều rộng của feature map.

Để chuyển đổi đặc trưng không gian 2D thành chuỗi đặc trưng tuần tự 1D phù hợp với các mô hình xử lý chuỗi ký tự, nhóm thực hiện quá trình reshape đặc trưng dọc theo chiều rộng của ảnh (trục thời gian). Với feature map dạng (B, C, H', W') , ta gộp chiều kênh đặc trưng và chiều cao lại với nhau, chuyển đổi về kích thước $(B, W', C \times H')$. Với thiết lập của nhóm, quá trình biến đổi này có dạng:

$$(B, 256, 4, 32) \rightarrow (B, 32, 1024)$$

Chuỗi đặc trưng đầu ra có độ dài thời gian $T = 32$, trong đó mỗi bước thời gian đại diện cho một dải ảnh thẳng đứng rộng 16 pixel trên ảnh gốc và được mô tả bởi một vector đặc trưng 1024 chiều.

Feature Reshape Process (Quy trình biến đổi đặc trưng không gian thành chuỗi)



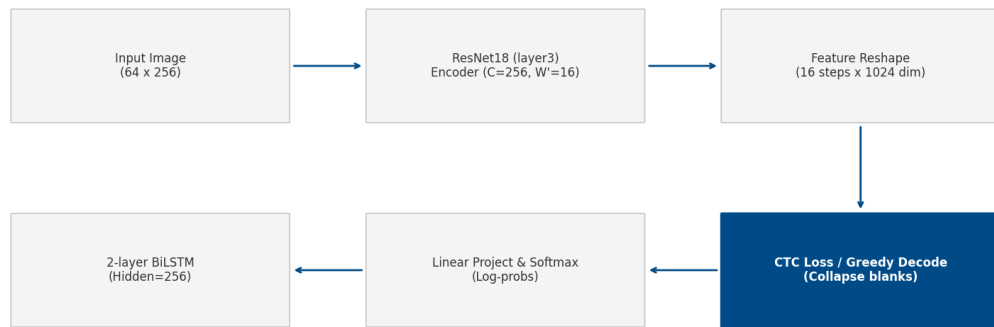
Hình 4.2: Minh họa quá trình chuyển đổi đặc trưng tích chập 2D thành chuỗi đặc trưng 1D

4.2. Kiến trúc CTC Baseline

Kiến trúc CTC Baseline được thiết kế làm mô hình đối chứng nhằm đánh giá hiệu quả của mô hình Attention đề xuất. Luồng xử lý của CTC Baseline được mô tả ngắn gọn như sau:

$$\text{Image} \rightarrow \text{ResNet18} \rightarrow \text{Reshape} \rightarrow \text{BiLSTM} \rightarrow \text{Linear} \rightarrow \text{CTC Loss}$$

CTC Baseline Architecture (Kiến trúc mô hình CTC)



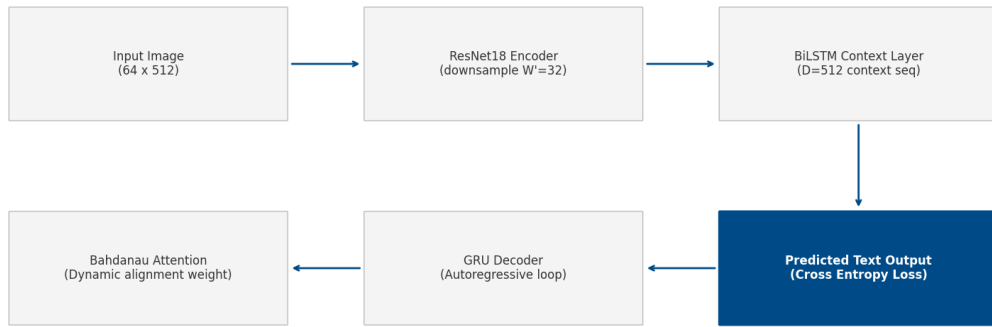
Hình 4.3: Kiến trúc chi tiết mô hình CTC Baseline

Sau khi feature map được reshape thành chuỗi đặc trưng $(B, 32, 1024)$, mô hình đưa chuỗi này qua mạng BiLSTM gồm 2 lớp với kích thước ẩn (hidden size) bằng 256 cho mỗi hướng. Nhờ cơ chế hoạt động hai chiều, BiLSTM tổng hợp ngữ cảnh xung quanh của từng dải ảnh từ cả hai hướng trái-phải và phải-trái, tạo ra vector đầu ra 512 chiều tại mỗi bước thời gian.

Vector ngữ cảnh này tiếp tục đi qua một lớp tuyến tính (Linear Projection) để ánh xạ lên không gian từ vựng ký tự. Nếu kích thước bộ từ vựng (bao gồm cả blank token) là V , đầu ra tại mỗi bước thời gian là một phân phối xác suất trên V ký tự. Hàm mất mát CTC Loss tối đa hóa tổng xác suất của tất cả các chuỗi căn chỉnh hợp lệ dẫn đến nhãn đúng. Khi suy diễn, mô hình sử dụng thuật toán CTC Greedy Decoding để chọn ký tự có xác suất lớn nhất tại mỗi bước thời gian, sau đó loại bỏ các ký tự trùng lặp liên tiếp và loại bỏ blank token để tạo ra kết quả cuối cùng.

4.3. Kiến trúc Attention Model

Mô hình đề xuất Attention Model sử dụng kiến trúc Encoder–Decoder tự hồi quy kết hợp cơ chế chú ý Bahdanau Attention để liên kết động giữa chuỗi đặc trưng ảnh và chuỗi ký tự đầu ra.

Attention Model Architecture (Kiến trúc mô hình Attention)**Hình 4.4: Kiến trúc chi tiết mô hình Attention**

Encoder của mô hình gồm mạng CNN ResNet18 và một khối BiLSTM Context Encoder (2 lớp, ẩn 256 mỗi hướng). Khối BiLSTM này giúp làm giàu ngữ cảnh toàn cục cho chuỗi đặc trưng ảnh trước khi đưa vào Bahdanau Attention, hỗ trợ tính toán trọng số attention chính xác hơn.

Tại Decoder, nhóm sử dụng GRUCell để sinh ra từng ký tự tại mỗi bước thời gian t . Ở bước giải mã t , cơ chế Bahdanau Attention tính toán điểm tương quan giữa trạng thái ẩn của decoder ở bước trước s_{t-1} và từng vector đặc trưng encoder h_s :

$$e_{t,s} = v_a^T \tanh(W_a s_{t-1} + U_a h_s)$$

Điểm số này được đưa qua hàm softmax để tạo thành trọng số chú ý $\alpha_{t,s}$, thể hiện mức độ mô hình tập trung vào vùng đặc trưng s của ảnh khi sinh ký tự tiếp theo. Vector ngữ cảnh c_t được tính bằng tổng có trọng số:

$$c_t = \sum_{s=1}^T \alpha_{t,s} h_s$$

Decoder nhận đầu vào là sự kết hợp (concat) của vector ngữ cảnh c_t và vector embedding của ký tự đã dự đoán ở bước $t-1$, cập nhật trạng thái ẩn và đưa qua một lớp tuyến tính để dự đoán phân phối xác suất của ký tự tiếp theo.

Trong quá trình huấn luyện, nhóm sử dụng kỹ thuật Teacher Forcing Decay. Ở các epoch đầu, mô hình sử dụng nhãn đúng từ tập dữ liệu làm đầu vào cho bước tiếp theo với xác suất 0.5 để giúp decoder học nhanh hơn. Xác suất này được giảm dần qua từng epoch để tiệm cận với kịch bản suy diễn thực tế (khi decoder phải tự sử dụng ký tự dự đoán của

chính mình ở bước trước làm đầu vào cho bước sau).

Khi suy diễn, mô hình hỗ trợ cả giải mã Greedy (chọn ký tự tốt nhất tại mỗi bước) và giải mã Beam Search (duy trì K đường đi có xác suất tích lũy cao nhất). Beam Search đóng vai trò quan trọng trong việc sửa lỗi dừng sớm (khi mô hình phát ra token kết thúc $\langle \text{eos} \rangle$ quá sớm) bằng cách cho phép xem xét các nhánh giải mã phụ có độ dài dài hơn nhưng điểm số tích lũy tốt.

4.4. So sánh và đối chiếu hai kiến trúc

Bảng 4.1 tóm tắt các điểm khác biệt cốt lõi giữa hai mô hình được triển khai trong dự án.

Bảng 4.1: So sánh kiến trúc CTC Baseline và Attention Model

Đặc điểm	CTC Baseline	Attention Model
Mạng trích xuất đặc trưng	ResNet18 (đến layer3)	ResNet18 (đến layer3)
Mô hình hóa chuỗi	2 lớp BiLSTM	2 lớp BiLSTM Context Encoder
Cơ chế căn chỉnh	Tự động căn chỉnh ngầm qua CTC Loss	Căn chỉnh mềm động qua Bahdanau Attention
Cơ chế giải mã	CTC Greedy Decoding	Greedy Decoding hoặc Beam Search
Mạng giải mã	Không có (Lớp tuyến tính chiếu trực tiếp)	GRU Decoder tự hồi quy
Hàm mất mát	CTC Loss	Cross Entropy Loss
Mối liên hệ giữa nhãn	Giả định độc lập có điều kiện giữa các ký tự	Tự hồi quy, phụ thuộc vào các ký tự đã sinh trước đó
Dung lượng checkpoint	khoảng 75 MB	khoảng 91 MB

CTC Baseline có cấu trúc đơn giản, tốc độ huấn luyện và suy diễn nhanh nhờ tính chất song song hóa. Tuy nhiên, mô hình này bị giới hạn bởi giả định độc lập có điều kiện giữa các ký tự dự đoán, do đó khó nắm bắt được ngữ cảnh ngôn ngữ của từ. Attention Model khắc phục được nhược điểm này bằng cách sử dụng decoder tự hồi quy để học mối liên hệ tuần tự giữa các ký tự, đồng thời hỗ trợ trực quan hóa attention map để phân tích trực quan hành vi của mô hình. Bù lại, Attention Model phức tạp hơn, huấn luyện chậm hơn và dễ bị hiện tượng trôi attention (attention drift) khi giải mã chuỗi dài.

Trong mã nguồn của dự án, các giải thuật giải mã được tách biệt hoàn toàn trong thư mục `src/inference` để tăng tính mô-đun hóa, giúp tránh trùng lặp mã nguồn giữa các script huấn luyện, kiểm thử và suy diễn thực tế.

CHƯƠNG 5. CÀI ĐẶT HỆ THỐNG

5.1. Môi trường phát triển

Hệ thống được phát triển và huấn luyện trên môi trường cục bộ bằng Python và PyTorch. Việc chạy local giúp nhóm chủ động kiểm soát quá trình chuẩn bị dữ liệu, huấn luyện, đánh giá và lưu checkpoint mô hình.

Bảng 5.1: Cấu hình môi trường thực nghiệm

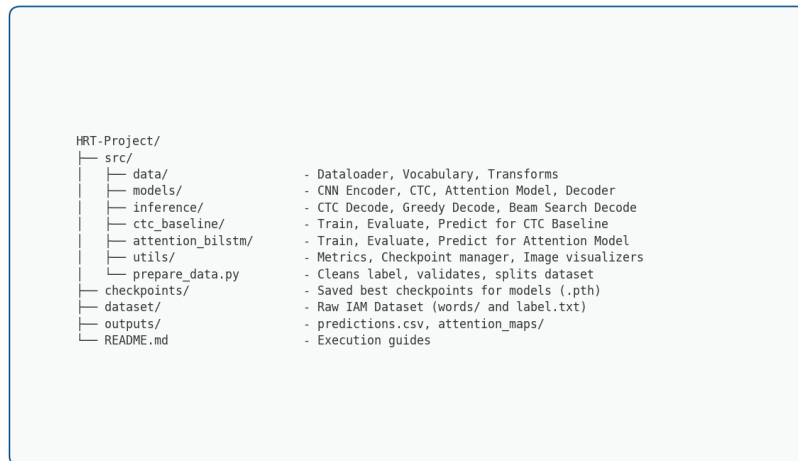
Thành phần	Thông tin sử dụng
Ngôn ngữ lập trình	Python 3.11
Framework học sâu	PyTorch, CUDA
Xử lý ảnh	OpenCV, Pillow
Xử lý dữ liệu	NumPy, Pandas
Tăng cường dữ liệu	Albumentations
Đánh giá chuỗi	editdistance
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB VRAM
CUDA	CUDA 13.0

Với cấu hình này, nhóm có thể huấn luyện hai mô hình trên tập IAM word-level sau khi làm sạch. Kích thước batch được lựa chọn tùy theo mức sử dụng bộ nhớ GPU và từng pipeline huấn luyện.

5.2. Tổ chức mã nguồn

Mã nguồn được tổ chức theo từng nhóm chức năng để dễ phát triển, kiểm thử và mở rộng. Cấu trúc tổng quát của project được minh họa trong Hình 5.1.

Codebase Directory Structure (Cấu trúc thư mục dự án)



Hình 5.1: Cấu trúc tổ chức mã nguồn dự án

Các thư mục chính trong project gồm:

- **src/data:** chứa các thành phần xử lý dữ liệu, bao gồm xây dựng từ vựng, đọc ảnh, encode nhãn và tiền xử lý ảnh.
- **src/models:** định nghĩa các thành phần mô hình như ResNet Encoder, CTC Baseline, Bahdanau Attention và GRU Decoder.
- **src/inference:** chứa các thuật toán giải mã đầu ra, gồm CTC Greedy Decode, Attention Greedy Decode và Beam Search.
- **src/utlis:** chứa các hàm tính metric, quản lý checkpoint và trực quan hóa attention map.
- **src/ctc_baseline** và **src/attention_bilstm:** chứa các script huấn luyện, đánh giá và dự đoán cho từng mô hình.

Phiên bản hiện tại của project là phiên bản v2. So với thử nghiệm ban đầu chỉ sử dụng chữ thường, phiên bản này mở rộng bộ từ vựng để hỗ trợ chữ thường, chữ hoa, chữ số và các token đặc biệt. Các checkpoint và câu lệnh trong chương này mặc định sử dụng phiên bản v2.

5.3. Chuẩn bị dữ liệu và huấn luyện

Trước khi huấn luyện, dữ liệu thô trong thư mục `dataset/` được xử lý bằng script `src/prepare_data.py`. Script này làm sạch nhãn, loại bỏ mẫu lỗi, chia dữ liệu theo tỷ lệ 80/10/10 và tạo các tệp CSV trong thư mục `data_processed/`.


```
# Chuẩn bị dữ liệu và chia train/val/test
python -m src.prepare_data

# Huấn luyện CTC Baseline v2
python -m src.ctc_baseline.train

# Huấn luyện Attention Model v2
python -m src.attention_bilstm.train
```

Trong quá trình huấn luyện, hệ thống lưu checkpoint dựa trên Word Accuracy tốt nhất trên tập validation. Khi kết quả validation tốt hơn kỷ lục trước đó, trọng số mô hình và trạng thái huấn luyện sẽ được lưu lại.

Checkpoint tốt nhất của phiên bản v2 được lưu trong các thư mục tương ứng:

- CTC Baseline: checkpoints/ctc_baseline_v2/best_ctc_baseline.pth
- Attention Model: checkpoints/attention_bilstm_v2/best_attention_model.pth

5.4. Đánh giá và suy diễn

Sau huấn luyện, mô hình được đánh giá trên tập test độc lập bằng cách nạp checkpoint đã lưu. Các chỉ số đánh giá gồm Word Accuracy, Character Error Rate (CER) và Normalized Edit Distance (NED).

```
# Thiết lập đường dẫn checkpoint
CTC_CKPT=checkpoints/ctc_baseline_v2/best_ctc_baseline.pth
ATT_CKPT=checkpoints/attention_bilstm_v2/best_attention_model.pth

# Đánh giá CTC Baseline v2
python -m src.ctc_baseline.evaluate --checkpoint $CTC_CKPT

# Đánh giá Attention Model v2 bằng Greedy Decode
python -m src.attention_bilstm.evaluate --checkpoint $ATT_CKPT

# Đánh giá Attention Model v2 bằng Beam Search
python -m src.attention_bilstm.evaluate --checkpoint $ATT_CKPT
↪ --beam
```

Để thử nghiệm trên ảnh chữ viết tay thực tế, nhóm sử dụng các script `predict.py`. Đầu ra dự đoán và các tệp kết quả được lưu trong thư mục `outputs/`.

```
# Thiết lập đường dẫn checkpoint và ảnh đầu vào
CTC_CKPT=checkpoints/ctc_baseline_v2/best_ctc_baseline.pth
ATT_CKPT=checkpoints/attention_bilstm_v2/best_attention_model.pth
```

```
IMG=test_images/sonb123.png

# Dự đoán ảnh thực tế bằng CTC Baseline
python -m src.ctc_baseline.predict --image $IMG --checkpoint
↪ $CTC_CKPT

# Dự đoán bằng Attention Model và lưu attention map
python -m src.attention_bilstm.predict --image $IMG --checkpoint
↪ $ATT_CKPT --save_attention

# Dự đoán bằng Attention Model với Beam Search
python -m src.attention_bilstm.predict --image $IMG --checkpoint
↪ $ATT_CKPT --beam
```

Với mô hình Attention, tùy chọn `-save_attention` cho phép lưu bản đồ attention nhằm trực quan hóa vùng ảnh mà decoder tập trung ở từng bước sinh ký tự. Đây là công cụ hữu ích để phân tích lỗi, ví dụ trường hợp mô hình dừng giải mã sớm hoặc bỏ sót ký tự ở cuối chuỗi.

5.5. Tổng kết chương

Chương này đã trình bày môi trường phát triển, cách tổ chức mã nguồn và các lệnh chính để chuẩn bị dữ liệu, huấn luyện, đánh giá và dự đoán. Toàn bộ pipeline được thiết kế tách biệt giữa hai mô hình CTC Baseline và Attention Model, giúp quá trình thực nghiệm và so sánh kết quả được thực hiện rõ ràng hơn.

CHƯƠNG 6. THỰC NGHIỆM VÀ KẾT QUẢ

6.1. Thiết lập thực nghiệm

Để thực hiện đánh giá công bằng, hai mô hình CTC Baseline và Attention Model được huấn luyện và kiểm thử trên cùng bộ dữ liệu IAM ở mức word-level với cấu hình Phase 2 (hỗ trợ đầy đủ 76 ký tự bao gồm chữ thường, chữ hoa, số và ký tự đặc biệt). Tập dữ liệu bao gồm 111,706 mẫu được phân chia thống nhất theo tỷ lệ 80/10/10: 89,364 mẫu huấn luyện, 11,170 mẫu kiểm định (validation) và 11,172 mẫu kiểm thử (test).

Mô hình được huấn luyện local với các tham số cấu hình tối ưu: batch size là 128, sử dụng thuật toán tối ưu Adam. Mạng ResNet18 trích xuất đặc trưng được freeze trong các epoch đầu tiên để ổn định các lớp phía sau, sau đó unfreeze để fine-tune với learning rate nhỏ hơn. Các chỉ số được sử dụng để đo lường hiệu năng là Word Accuracy (WA), Character Error Rate (CER) và Normalized Edit Distance (NED).

6.2. Kết quả thực nghiệm trên tập kiểm thử IAM

Kết quả thực nghiệm của hai mô hình trên 11,172 mẫu kiểm thử độc lập (Phase 2) được tổng hợp trong Bảng 6.1.

Bảng 6.1: Kết quả so sánh hiệu năng của hai mô hình trên IAM Test Set (Phase 2)

Mô hình	Giải mã	Word Accuracy	CER	NED
CTC Baseline	CTC Greedy	79.95%	8.25%	93.13%
Attention Model	Greedy Decode	82.67%	7.79%	93.32%
Attention Model	Beam Search (K=2)	82.84%	7.74%	93.35%

Bảng số liệu cho thấy Attention Model tốt hơn CTC Baseline ở cả ba chỉ số đánh giá. Word Accuracy của Attention Model với giải mã Greedy đạt 82.67% và tăng lên 82.84% khi sử dụng giải mã Beam Search (với độ rộng chùm K=2), cao hơn đáng kể so với mức 79.95% của CTC Baseline. Chỉ số Character Error Rate (CER) của Attention Model cũng giảm xuống mức 7.74% so với 8.25% của baseline. Chỉ số Normalized Edit Distance (NED) của hai mô hình tương ứng là 93.35% và 93.13%.

Kết quả thực nghiệm này phản ánh sự cải tiến có xuất hiện khi tích hợp bộ giải mã tự hồi

quy GRU kết hợp cơ chế Bahdanau Attention. So với Phase 1 trước đây (chỉ hỗ trợ 26 chữ cái viết thường và đạt WA 83.09% trên Attention), kết quả ở Phase 2 tuy thấp hơn một chút do không gian từ vựng lớn hơn gấp gần 3 lần (76 ký tự so với 26 ký tự) nhưng có giá trị thực tiễn cao hơn nhiều vì giữ nguyên chữ hoa, chữ số và dấu câu thực tế.

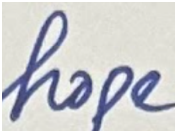
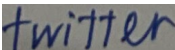
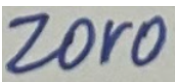
6.3. Phân tích kết quả và kiểm thử thực tế

Từ kết quả định lượng, Attention Model cho hiệu năng cao hơn CTC Baseline trên tập kiểm thử IAM. Sự khác biệt này đến từ việc Attention Model sử dụng bộ giải mã tự hồi quy, trong đó ký tự được sinh ở bước hiện tại có liên hệ với các ký tự đã sinh trước đó. Nhờ đó, mô hình có khả năng tận dụng ngữ cảnh chuỗi tốt hơn so với cách dự đoán độc lập theo từng timestep của CTC.

Tuy nhiên, mức cải thiện giữa hai mô hình không quá lớn và cần được nhìn nhận trong phạm vi bộ dữ liệu IAM. CTC Baseline vẫn là một hướng tiếp cận hiệu quả do có kiến trúc đơn giản hơn, quá trình huấn luyện ổn định và tốc độ suy diễn nhanh. Trong khi đó, Attention Model có khả năng biểu diễn linh hoạt hơn nhưng phụ thuộc nhiều hơn vào chất lượng giải mã và phân phối dữ liệu huấn luyện.

Bảng 6.2 trình bày một số ví dụ dự đoán tiêu biểu nhằm minh họa sự khác biệt giữa hai mô hình trong các trường hợp đúng, sai nhẹ hoặc có nhầm lẫn ký tự.

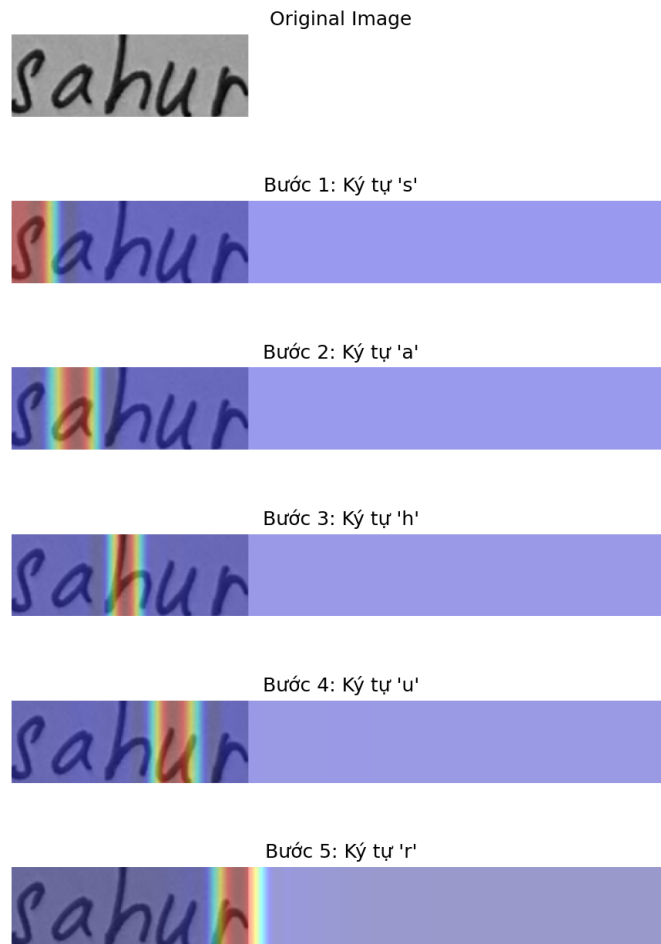
Bảng 6.2: Một số ví dụ dự đoán tiêu biểu của hai mô hình

Ảnh mẫu	Nhãn đúng	CTC đoán	dự đoán	Nhận xét
	hope	hope	hope	Cả hai mô hình đều nhận dạng chính xác.
	scripting	scriping	scripting	Attention nhận dạng đúng; CTC bị thiếu ký tự “t” do dính nét.
	twitter	twitten	twitter	Attention nhận dạng đúng; CTC nhầm “r” thành “n” do nét viết gần giống.
	zoro	2or0	20r0	Cả hai đều sai nhẹ do nhầm lẫn chữ cái và chữ số, ví dụ “z” → “2”, “o” → “0”.

Các ví dụ trên cho thấy Attention Model thường có lợi thế trong những trường hợp cần khai thác quan hệ giữa các ký tự liên tiếp. Tuy vậy, mô hình vẫn có thể nhầm lẫn khi chữ

viết bị dính nét, ký tự có hình dạng gần giống nhau hoặc ảnh đầu vào khác nhiều so với phân phối của tập IAM.

Đối với mô hình Attention, nhóm cũng sử dụng attention map để quan sát vùng ảnh mà decoder tập trung tại từng bước sinh ký tự. Hình 6.1 minh họa một trường hợp mô hình dịch chuyển vùng chú ý theo chiều trái sang phải của ảnh, tương ứng với quá trình sinh chuỗi ký tự.



Hình 6.1: Ví dụ trực quan hóa attention map của mô hình Attention

Attention map giúp việc phân tích kết quả trở nên trực quan hơn, đặc biệt khi cần quan sát mô hình đang tập trung vào vùng ảnh nào trong quá trình dự đoán. Trong một số trường hợp khó, vùng chú ý có thể chưa hoàn toàn khớp với vị trí ký tự thực tế, nhất là với ảnh chữ viết tay tự chụp, ký tự ít xuất hiện hoặc phần nền/padding chiếm tỷ lệ lớn. Đây là một hạn chế cần được cải thiện nếu muốn mở rộng hệ thống sang dữ liệu thực tế ngoài IAM.

Khi thử nghiệm thêm với ảnh viết tay ngoài tập dữ liệu, nhóm nhận thấy mô hình vẫn chịu ảnh hưởng của hiện tượng lệch phân phối dữ liệu. Các yếu tố như độ nghiêng chữ, độ dày nét bút, cách crop ảnh, ánh sáng và sự xuất hiện của chữ số hoặc ký tự đặc biệt có thể làm kết quả suy giảm so với tập kiểm thử IAM. Do đó, kết quả trên IAM cho thấy tiềm năng của mô hình, nhưng chưa đủ để kết luận hệ thống có thể hoạt động ổn định trên mọi ảnh chữ viết tay thực tế.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận và kết quả đạt được

Trong phạm vi bài tập lớn, nhóm đã thiết lập thành công một hệ thống nhận dạng chữ viết tay mức độ từ đơn lẻ chạy cục bộ sử dụng thư viện PyTorch. Chúng tôi đã xây dựng một pipeline học sâu khép kín và tự động từ khâu đọc dữ liệu, làm sạch nhãn, tiền xử lý và tăng cường ảnh, huấn luyện mô hình cho đến kiểm thử độc lập và suy diễn thực tế.

Nhóm đã cài đặt hoàn chỉnh và so sánh hai kiến trúc: mô hình đối chứng CTC Baseline (ResNet18 + BiLSTM + CTC Loss) và mô hình cải tiến Attention Model (ResNet18 + Context BiLSTM + Bahdanau Attention + GRU Decoder).

Kết quả thực nghiệm trên tập kiểm thử IAM cho thấy mô hình Attention đạt độ chính xác từ tốt hơn so với CTC Baseline (82.84% WA ở chế độ Beam Search so với 79.95% của CTC Baseline). Việc tích hợp thêm khối BiLSTM Context Encoder đóng vai trò lớn trong việc giúp mô hình Attention đạt kết quả tối ưu. Không chỉ dừng lại ở các chỉ số định lượng, nhóm cũng phát triển công cụ trực quan hóa attention map để tăng tính giải thích cho mô hình và cung cấp các script suy diễn linh hoạt phục vụ cho quá trình triển diễn hệ thống.

7.2. Hạn chế của hệ thống

Mặc dù đạt được những kết quả khả quan trên tập dữ liệu chuẩn, hệ thống vẫn tồn tại các hạn chế cụ thể sau:

- **Phạm vi nhận dạng:** Hệ thống mới chỉ nhận dạng ở mức độ từ đơn lẻ (word-level). Dự án chưa hỗ trợ nhận dạng dòng văn bản (line-level) hay toàn bộ trang tài liệu, vốn đòi hỏi thêm các kỹ thuật phát hiện và phân đoạn vùng chữ phức tạp hơn.
- **Khả năng tổng quát hóa:** Mô hình bị phụ thuộc nặng nề vào phân phối của bộ dữ liệu IAM. Khi suy diễn trên ảnh thực tế tự viết có chất lượng ánh sáng kém, góc chụp không thẳng hoặc nét chữ cá nhân quá dày/mỏng, độ chính xác giảm sút đáng kể.
- **Lỗi dừng sớm của Attention:** Khi giải mã các chuỗi ký tự chứa số hoặc ký tự đặc

biệt ít xuất hiện trong tập train (như từ "sonb123"), mô hình giải mã Greedy dễ bị lỗi sinh ra token kết thúc `<eos>` quá sớm do sự thiếu tự tin từ việc mất cân bằng dữ liệu.

- **Sự phụ thuộc ngôn ngữ:** Thuật toán Beam Search trong dự án mới chỉ giải mã dựa trên điểm xác suất cục bộ của mô hình thị giác, chưa tích hợp với một mô hình ngôn ngữ ngoài (Language Model) để hiệu chỉnh chính tả từ vựng một cách thông minh.

7.3. Hướng phát triển tương lai

Để khắc phục các hạn chế trên, hướng nghiên cứu tiếp theo của dự án có thể tập trung vào:

- **Cải thiện tiền xử lý ảnh thực tế:** Bổ sung các bước cân bằng ánh sáng, loại bỏ bóng mờ, deskew (chỉnh thẳng độ nghiêng nét chữ) và tự động crop sát biên để giảm thiểu tác động của domain shift.
- **Áp dụng Mask cho Attention:** Bổ sung attention padding mask trong quá trình tính toán trọng số chú ý để ngăn không cho cơ chế attention tập trung lệch sang vùng đệm trắng ở cuối ảnh.
- **Kết hợp mô hình ngôn ngữ:** Tích hợp một mô hình ngôn ngữ dạng n-gram hoặc mạng nơ-ron ngoài vào bước Beam Search giải mã của mô hình Attention để sửa lỗi chính tả tự động.
- **Mở rộng dữ liệu và kiến trúc:** Bổ sung thêm các nguồn dữ liệu chữ viết tay tiếng Việt hoặc kiểu chữ đa dạng, đồng thời nghiên cứu thử nghiệm các kiến trúc mạnh mẽ hơn như Transformer-based HTR.

TÀI LIỆU THAM KHẢO

- [1] U. V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 369–376, 2006.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *International Conference on Learning Representations*, 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [5] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp. 1724–1734, 2014.
- [7] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [8] PyTorch Contributors, “PyTorch Documentation,” Available: <https://pytorch.org/docs/stable/index.html>.
- [9] PyTorch Contributors, “Torchvision Models and Pre-trained Weights,” Available: <https://pytorch.org/vision/stable/models.html>.

- [10] Alumentations Team, “Alumentations Documentation,” Available: <https://alumentations.ai/docs/>.
- [11] OpenCV Team, “OpenCV Documentation,” Available: <https://docs.opencv.org/>.
- [12] IAM Handwriting Database, “IAM Handwriting Database Official Website,” Available: <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>.